

Critique of Recent Vision-Language Papers and a Local Reproduction of BLIP-2

Anonymous ACL submission

Abstract

This report follows the term-paper workflow of reading, critiquing, implementing, and evaluating recent NLP and multimodal papers. I study three candidate papers: BLIP-2, LLaVA, and VisionLLM v2. The critique focuses on architecture, supervision strategy, compute requirements, openness, and reproducibility under student-level hardware constraints. Based on that analysis, I select BLIP-2 as the implementation target because its modular design offers the strongest downscaling story: a frozen image encoder, a frozen language model, and a relatively small trainable bridge. I then build a local BLIP-2 reproduction on a single NVIDIA RTX 3070 8GB GPU using the official LAVIS codebase, reduced COCO Karpathy subsets, CLIP-L, and OPT-350M. The reproduction successfully completes all three major stages of the BLIP-2 pipeline: stage-1 representation learning, stage-2 generative alignment, and caption fine-tuning, and it produces valid checkpoints and caption outputs for each stage. On an office-scale 50k/1k/1k COCO Karpathy subset, the best completed local validation snapshot reaches BLEU-4 of 11.13 and CIDEr of 37.57 when reported on the same paper-style scale as BLIP-2's COCO table, compared with BLIP-2 ViT-g OPT2.7B's reported BLEU-4 of 43.7 and CIDEr of 145.8 on the Karpathy COCO test split. A larger 100k student-scale run was also completed with a longer 10 / 10 / 15 schedule, but its best validation checkpoint peaked early at epoch 2 with BLEU-4 8.57 and CIDEr 25.34 and therefore did not surpass the earlier 50k office-scale best. The main conclusion is that BLIP-2 is reproducible at the pipeline level on student hardware, but not reproducible at paper-level caption quality without large-scale data, compute, and backbone capacity.

1 Introduction

Recent vision-language models combine large pre-trained visual backbones, large language models, and task-specific multimodal alignment strategies.

Their capabilities have improved rapidly across captioning, question answering, grounding, and assistant-style interaction, but the cost of reproducing paper-level systems has also risen sharply. For a student project, this creates a useful scientific question: which parts of a modern multimodal result are genuinely reproducible on commodity hardware, and which parts depend primarily on scale?

The assignment requires two equally important parts: a critique of at least three papers and a non-trivial implementation with evaluation. I therefore treat this report as both a comparative review and a reproduction study. The three papers I selected are BLIP-2 (Li et al., 2023), LLaVA (Liu et al., 2023), and VisionLLM v2 (Wu et al., 2024). All three are influential multimodal systems, but they differ sharply in how they bridge vision and language, how much supervision they require, and how practical they are to reproduce.

My final implementation target is BLIP-2. The reason is not that it is easy in an absolute sense. Rather, it is the one whose core methodological idea still survives aggressive downscaling: freeze a strong image encoder and language model, and train a relatively small bridge between them. This makes it much more realistic than VisionLLM v2, whose unified multi-decoder design depends on a very large engineering and compute budget, and safer than reproducing LLaVA under a single 8GB GPU when the goal is not only to imitate assistant behavior, but to study the staged training method itself.

The central claim of this report is that BLIP-2 is reproducible at the *pipeline* level on student hardware, but not reproducible at the *performance* level without paper-scale data, compute, and backbones. The project therefore asks two questions at once: whether the official training path can be made to run end to end in a constrained local environment, and how far the resulting system remains from the published captioning performance after aggressive

reductions in scale.

More specifically, the report contributes four things. First, it provides a comparative critique of three recent multimodal papers through the lens of student-scale reproducibility. Second, it documents a local end-to-end BLIP-2 reproduction built from the official LAVIS codebase rather than from a simplified reimplementation. Third, it records the engineering changes required to make that pipeline run reliably on a single non-distributed Windows workstation. Fourth, it analyzes the resulting metric gap as an empirical finding about scaling, not merely as a missing optimization detail. This framing matters because the final value of the project lies as much in the methodology and artifact trail as in the absolute caption score.

2 Paper Critique

2.1 BLIP-2

BLIP-2’s main contribution is the Querying Transformer (Q-Former), which serves as a lightweight bottleneck between a frozen image encoder and a frozen large language model (Li et al., 2023). The paper’s strongest idea is its two-stage training strategy. First, it learns language-relevant visual representations from a frozen image encoder. Second, it aligns those learned visual queries to a frozen language model for generation. This decomposition is elegant because it turns a difficult end-to-end multimodal optimization problem into a modular bridge-learning problem.

The strengths of BLIP-2 are clear. It is compute-aware relative to prior frontier systems, it is open-source through LAVIS, and it performs well across zero-shot VQA, captioning, and retrieval. It is also conceptually clean: the model architecture directly reflects the hypothesis that a small trainable bridge can leverage powerful frozen unimodal backbones. This modularity is especially important for reproduction work because it provides a plausible path for downscaling: if the core bridge-learning idea is valid, then some version of it should still function under smaller backbones and smaller datasets.

BLIP-2 is also interesting in relation to adjacent work. Compared with Flamingo, which interleaves visual tokens and language modeling at large scale (Alayrac et al., 2022), BLIP-2 makes the cross-modal interface much more explicit through the Q-Former bottleneck. Compared with the original BLIP framework, which trained a unified encoder-decoder family for understanding and generation

(Li et al., 2022), BLIP-2 leans harder into frozen-backbone reuse. For a reproduction study, this distinction is important: a bottlenecked bridge is easier to reason about experimentally than a more fully coupled multimodal stack.

Its main weakness is that “compute-efficient” is still relative. The paper still uses 129M pre-training images, 250k first-stage steps, and 80k second-stage steps. The appendix states that even the largest model still needs a 16-A100 machine for less than six days in stage 1 and less than three days in stage 2 (Li et al., 2023). So BLIP-2 is much more realistic than some alternatives, but it is still far from trivial to reproduce faithfully, and the paper’s strongest reported numbers should be understood as the product of substantial scale rather than merely elegant architecture.

2.2 LLaVA

LLaVA’s key idea is visual instruction tuning: connect a vision encoder to a Vicuna-like LLM and then tune the model on GPT-generated multimodal instruction data (Liu et al., 2023). Conceptually, it is attractive because it directly targets assistant-like behavior rather than only representation learning. The paper is influential because it helped shift multimodal work toward instruction-following interfaces and open-source chat-style systems.

LLaVA’s strengths are simplicity and task relevance. It is easier to explain than a multi-stage multimodal pre-training framework, and its outputs align well with what users expect from a general visual assistant. It is also comparatively open and reproducible. The appendix reports training on 8 A100 GPUs, with pretraining completing in about 4 hours and instruction tuning in about 10 hours (Liu et al., 2023). That is still a serious budget, but much smaller than very large proprietary systems.

Its main weakness is that its strongest behavior depends heavily on synthetic GPT-generated data and instruction tuning quality. In other words, it is powerful as an assistant, but its gains are tied to the data pipeline and alignment setup as much as to the underlying architectural novelty. For a class project on limited hardware, it is reproducible only if moderate multi-GPU compute is available.

2.3 VisionLLM v2

VisionLLM v2 aims at a much more ambitious goal: a single end-to-end generalist multimodal model that supports hundreds of vision and vision-language tasks, including perception, understand-

ing, generation, and editing (Wu et al., 2024). Its “super link” mechanism routes task-specific information from the central MLLM to specialized decoders such as Grounding DINO, UniPose, and diffusion-based image generators.

The paper’s strength is scope. It is not merely a VQA or captioning model; it tries to unify diverse output types inside one system. The model is also thoughtful about task conflicts and multi-stage training, which are real problems in generalist multimodal systems.

The weakness is reproducibility. The appendix reports a three-stage regime using 64 A100 GPUs for stage 1 and 128 A100 GPUs for stages 2 and 3, with approximately 5, 3, and 10 days of training respectively (Wu et al., 2024). That scale, combined with many decoders, curated datasets, and dependency-heavy tooling, makes it unrealistic for this assignment unless one already has institutional cluster access and substantial engineering time.

2.4 Comparison and Paper Choice

Table 1 summarizes the main comparison. The most important conclusion is that BLIP-2 offers the best balance between novelty, non-triviality, openness, and downscalable reproducibility.

3 Reproduction Study

3.1 Objective and Scope

The goal of the implementation was not to exactly match the paper’s full training regime. That would have been impossible on the available hardware. Instead, the goal was to reproduce the *core BLIP-2 pipeline* from scratch in a scaled local setting:

- stage-1 vision-language representation learning,
- stage-2 vision-to-language generative alignment with OPT,
- caption fine-tuning on COCO-style data.

The target of reproduction was therefore methodological fidelity under aggressive scaling constraints, not paper-scale replication. A successful outcome meant that the staged BLIP-2 training pipeline would run end to end, produce valid checkpoints, generate caption outputs, and support reproducible evaluation on a subset-matched caption benchmark. An unsuccessful outcome would not necessarily mean a broken implementation; it could also mean that the pipeline runs, but the resulting

model remains qualitatively or quantitatively weak because the reproduction differs too much from the original paper in scale.

The local implementation uses the official LAVIS BLIP-2 code path, but with carefully chosen reductions and fixes so that the full pipeline can run on a single RTX 3070 8GB GPU. In that sense, this project is both a model reproduction and a systems study of what it takes to make a research codebase operate under local, single-GPU, non-distributed conditions.

3.2 BLIP-2 Method Overview

BLIP-2 is built around a simple but powerful architectural separation. A pretrained vision encoder extracts visual features, a pretrained language model generates text, and a trainable Querying Transformer (Q-Former) acts as the bottleneck between them (Li et al., 2023). The image encoder and language model remain frozen, while the Q-Former learns a compact set of query-conditioned visual representations that are useful to the language model. This design is important for reproducibility because it avoids the need to finetune all parameters of a large multimodal system and concentrates learning in the bridge.

More concretely, let an image x be encoded by a frozen vision encoder E_v into visual features $V = E_v(x)$. The Q-Former maintains a fixed set of learned query tokens $q_1, \dots, q_M$ and applies cross-attention from those queries into V , producing query-conditioned outputs $Z = Q_\theta(V)$, where θ denotes the trainable bridge parameters. In the local runs, $M = 32$ queries. A linear projection then maps Z into the embedding space expected by the frozen language model. In the original paper, this mechanism is meant to preserve the strengths of both pretrained backbones while learning only the minimal cross-modal interface needed for downstream generation and understanding. For a reproduction study, that architectural modularity is the main reason the method remains scientifically meaningful after severe downscaling.

The training procedure has three distinct stages in the local reproduction. Stage 1 learns language-relevant visual representations from paired image-text data. In the LAVIS BLIP-2 implementation, this stage combines image-text contrastive loss, image-text matching loss, and a language modeling loss:

$$\mathcal{L}_{\text{stage1}} = \mathcal{L}_{\text{itc}} + \mathcal{L}_{\text{itm}} + \mathcal{L}_{\text{lm}}.$$

Paper	Core idea	Main strengths	Main weaknesses	Reproducibility judgment
BLIP-2	Two-stage Q-Former bridge between frozen image encoder and frozen LLM	Clean modular design, strong results, open-source LAVIS implementation, best downscaling story	Still depends on huge pre-training scale; paper setting is far beyond student hardware	Best choice for a scaled reproduction of the core method
LLaVA	Visual instruction tuning of a vision encoder plus Vicuna-style LLM on GPT-generated multimodal instructions	Simple architecture, strong assistant behavior, strong community adoption, moderate openness	Performance depends heavily on instruction data pipeline; still needs multi-GPU training for faithful reproduction	Good second choice with 8x A100-level access
VisionLLM v2	End-to-end generalist MLLM with routing tokens, super-link queries, and multiple task-specific decoders	Broadest task coverage, strong generalist ambition, perception plus generation in one framework	Extremely high engineering and compute burden; many decoders and datasets; hard to ablate cleanly	Not realistic for this assignment on local hardware

Table 1: Comparative critique of the three selected papers.

The contrastive objective $\mathcal{L}_{\text{itc}}$ encourages matched image and caption representations to be closer than mismatched pairs in the batch. The matching objective $\mathcal{L}_{\text{itm}}$ trains a binary classifier over fused image-text features to distinguish aligned from non-aligned pairs after cross-modal interaction. The language modeling term $\mathcal{L}_{\text{lm}}$ encourages the visual queries to support caption-token prediction under teacher forcing. These losses serve different roles: contrastive loss improves global alignment, matching loss refines pairwise discrimination, and language modeling loss encourages generative usefulness.

Stage 2 aligns the learned Q-Former outputs with a frozen language model. In this project, the local target language model is facebook/opt-350m rather than the much larger OPT variants used for stronger paper results. If W denotes the learned projection from query outputs into the language-model embedding space and $y = (y_1, \dots, y_T)$ denotes the target caption tokens, stage 2 optimizes an autoregressive objective of the form

$$\mathcal{L}_{\text{stage2}} = - \sum_{t=1}^T \log p(y_t \mid y_{<t}, WZ),$$

where the language model is conditioned on the projected visual queries but its own weights remain frozen. The practical role of stage 2 is to teach the bridge how to condition a language model on visual representations in a generative setting. Caption finetuning then adapts that image-conditioned generative path to the COCO captioning task itself. In the local caption configs, this final stage uses

the prompt “a photo of” and keeps the same frozen-backbone structure while optimizing specifically for caption generation on the reduced COCO split. This final stage is where the reproduction is judged most directly against the paper’s published captioning behavior, even though the local setup remains much smaller than the original.

3.3 Experimental Environment

The reproduction was carried out on a single NVIDIA RTX 3070 8GB GPU in a Windows desktop environment. The software environment used the workspace virtual environment, PyTorch with CUDA support, Java 8 for caption-evaluation dependencies, and the official Salesforce LAVIS implementation as the base code path. The project also relied on a local mirror of the COCO annotation and image cache under the standard LAVIS directory structure. This environment matters because many of the engineering decisions in the project were direct responses to local hardware limits, Windows-specific runtime behavior, or assumptions in the upstream codebase about distributed Linux training.

The hardware budget strongly constrained all major design choices. The local system could not support paper-scale backbones, paper-scale image resolution, or paper-scale optimization budgets. As a result, the reproduction used CLIP ViT-L instead of ViT-g, OPT-350M instead of OPT-2.7B, image resolution 224 instead of 364, and reduced COCO Karpathy subsets instead of the paper’s large pre-training corpora. These reductions were not minor implementation details; they define the scientific

meaning of the final comparison.

The local environment also implied a specific optimization regime. All runs used automatic mixed precision, a cosine-style schedule with linear warmup, and gradient accumulation to emulate a larger effective batch size than device memory would otherwise allow. For example, the current 100k long-run stage-1 configuration uses batch size 4 with accumulation 8, giving an effective batch of 32 samples per optimizer step, while stage 2 and caption finetuning use batch size 2 with accumulation 8, giving an effective batch of 16. This is a good example of the difference between model design and systems design: the conceptual BLIP-2 method comes from the paper, but the practical ability to train it on 8GB VRAM depends on careful low-level choices in the run configuration.

The environment also shaped the operational workflow. Because the project ran in a single-process desktop setting rather than on a distributed Linux cluster, the code had to tolerate missing distributed process groups, local filesystem paths, long-running interactive PowerShell sessions, and occasional interruption or restart of multi-hour jobs. In other words, the environment influenced not only runtime limits but also the structure of the engineering solution.

3.4 Dataset Construction and Evaluation Protocol

The local experiments use the COCO Karpathy caption splits as the base captioning benchmark (Lin et al., 2014). Rather than using the full paper-scale training mixture, I constructed smaller deterministic subsets for local runs. The first complete local baseline used a 10k / 1k / 1k train/validation/test split. The strongest completed run by caption metrics uses an office-scale 50k / 1k / 1k subset. A larger student-scale 100k / 1k / 1k run was also completed under a longer training schedule.

The subset construction process matters because captioning quality is highly sensitive to both dataset scale and evaluation consistency. Subset JSON files were generated from the original Karpathy annotations with deterministic shuffling, and only the unique images referenced by those subset annotations were staged into the local COCO image cache. This process allowed the project to scale data upward over time while preserving a consistent evaluation protocol across runs. It also made it possible to distinguish the effect of additional data from the effect of unrelated code or architecture

Split	Train	Val	Test
Baseline local	10,000	1,000	1,000
Office scale	50,000	1,000	1,000
Student scale	100,000	1,000	1,000

Table 2: Subset scales used in the local reproduction study.

changes.

Evaluation also required a custom path. The stock LAVIS caption evaluation assumes the standard full COCO ground-truth file and uses a metric path that was unstable in the local Windows environment, especially for METEOR. For that reason, the final checkpoint-producing caption runs were executed with `report_metric=false`, and BLEU/CIDEr were computed afterward from saved `val_epoch*.json` prediction files using `pycocoevalcap` on a subset-matched COCO-format ground-truth file. This design made the metric computation reproducible and ensured that prediction and ground-truth image IDs were aligned exactly.

This evaluation design introduces an important interpretive distinction. The local scores are internally consistent across the 10k, 50k, and 100k experiments because they share the same reduced validation subset protocol and the same offline metric code path. However, they are not numerically interchangeable with the paper’s official Karpathy test-set results. In the final analysis, therefore, the comparison to BLIP-2 should be read as a scale-gap argument rather than as a strict leaderboard comparison.

3.5 Experimental Setup

The final setup is intentionally much smaller than the paper’s. I use the workspace virtual environment, the official LAVIS implementation, and reduced COCO Karpathy subsets rather than paper-scale pretraining corpora. The strongest completed result comes from an office-scale reduced COCO subset of 50,000 train, 1,000 validation, and 1,000 test samples. The local model uses CLIP-L as the vision encoder, OPT-350M as the language model, image resolution 224, 32 learned query tokens, and gradient accumulation 8 throughout. Micro-batch size depends on the stage: the completed office run uses batch size 2 for stage 1 and batch size 1 for stage 2 and caption finetuning, while the completed 100k long run increases this to batch size 4 for stage 1 and batch size 2 for stage 2 and caption-

ing. The completed office runs use 3 epochs for stage 1, 3 epochs for stage 2, and a 5-epoch caption schedule. The student-scale long run expands this further to a 100k / 1k / 1k split and a 10 / 10 / 15 schedule.

Across runs, the architecture itself is intentionally stable. The main controlled variables are dataset size, epoch budget, and checkpointing policy. This was a deliberate experimental choice. On a constrained machine, changing the backbone, image resolution, training objective, and data scale at the same time would make the final outcome difficult to interpret. By keeping CLIP-L, OPT-350M, 32 query tokens, and the same basic LAVIS training path fixed, the study isolates whether additional data and longer schedules push the local system in the expected direction.

Table 3 shows why the local run should be interpreted as a scaled reproduction rather than a full replication.

The local compute budget was measured from output directory creation and final-write timestamps:

- Initial complete local baseline (10k path): 1h 28m 53s
- Completed office caption polish rerun alone: 5h 59m 50s
- Current 100k stage-1 long run: 16h 20m 39s

3.6 Implementation Details

Although the local run is small relative to the paper, the implementation was non-trivial. Beyond setting up data and configs, I had to patch several parts of the official code path so that BLIP-2 could train end to end in a single-process Windows environment.

The most important fixes were:

- overriding LAVIS cache-path assumptions so the reduced local COCO cache was actually used,
- adding non-distributed guards for collective operations and barriers in single-GPU runs,
- fixing a caption-dataset image_id path that incorrectly assumed retrieval-style tensors,
- patching the OPT bridge so that OPT-350M projects into the actual token-embedding dimension rather than the hidden size,

- generating a reduced COCO validation ground-truth file so evaluation used the same image subset as the predictions,

- disabling local Windows METEOR scoring in the final checkpoint-producing caption run because the Java subprocess remained unstable.

Two engineering details deserve special emphasis. First, the local OPT-350M configuration was not a drop-in replacement for the larger OPT variants assumed by the stock BLIP-2 code path. The Q-Former bridge had to project into OPT’s actual token-embedding dimension rather than the hidden-size path assumed in the default code, or stage 2 would fail. Second, evaluation had to be reconstructed as a two-step process: save predictions during training, then score them offline against a subset-matched ground-truth file. Without that change, the local system would not have produced reliable caption metrics.

The implementation can be understood as four classes of intervention: environment compatibility, training-path correctness, evaluation correctness, and long-run orchestration. Environment compatibility covered dependency pinning, Java availability, and Windows-specific import behavior. Training-path correctness covered distributed guards and the OPT embedding-dimension fix. Evaluation correctness covered subset-matched ground-truth generation and the decision to move metric scoring offline. Long-run orchestration covered checkpoint frequency, resume hooks, and automatic stage chaining. Framing the patches this way is useful because it shows that the work was not random debugging; it was a systematic effort to restore the assumptions that the upstream code expected from a cluster setting.

The implementation also evolved toward a more robust checkpointing strategy over time. Early successful runs proved that the full pipeline could execute, but they were not ideal for selecting the best epoch because late-stage checkpoints were not always preserved. Later caption runs adopted per-epoch checkpoint saving, and the current long-run student-scale workflow adds explicit resume support and a top-level pipeline script that automatically chains stages by discovering the newest produced checkpoint. That automation is part of the scientific artifact, not just a convenience feature, because it supports multi-day local runs under unstable or interruptible conditions.

Dimension	BLIP-2 paper configuration	Local reproduction
Pre-training data	129M image-text pairs from COCO, VG, CC3M, CC12M, SBU, and LAION subsets (Li et al., 2023)	Reduced COCO Karpathy subsets only (10k, 50k, and 100k scaling paths)
Bridge training schedule	250k steps stage 1, 80k steps stage 2 (Li et al., 2023)	Completed runs: 3 epochs stage 1 and 3 epochs stage 2 on 50k; completed long run: 10 epochs stage 1 and 10 epochs stage 2 on 100k
Caption fine-tuning schedule	5 epochs, batch size 256, image resolution 364 (Li et al., 2023)	Completed office run: 5 epochs, batch size 1 with grad accumulation 8, image resolution 224; completed long run: 15 caption epochs on 100k with batch size 2 and grad accumulation 8
Vision encoder	ViT-g/14 in the reported OPT2.7B captioning result (Li et al., 2023)	CLIP ViT-L
Language model	OPT-2.7B for the best reported OPT caption result (Li et al., 2023)	OPT-350M
Compute	Largest model pre-training on a 16-A100(40G) machine, <6 days first stage and <3 days second stage (Li et al., 2023)	One RTX 3070 8GB; completed office and 100k student-scale runs executed locally across multi-day desktop sessions

Table 3: Original paper configuration versus the local reproduction.

Another practical issue appeared during image staging for the 100k run. The dataset downloader used concurrent network fetches and progress accounting, but individual HTTP requests could stall indefinitely because the underlying request path lacked an explicit timeout. In practice, this meant that image staging could appear “stuck” even after all required images had already been downloaded. This episode is representative of the broader project: a large part of the work was not about inventing a new model, but about making an existing research pipeline reliable under local conditions.

3.7 Training Procedure

The project progressed through a sequence of increasingly strong experiments. The first complete local baseline used a 10k / 1k / 1k split and a 1 / 1 / 1 schedule for stage 1, stage 2, and caption finetuning. The role of this experiment was primarily methodological: prove that the BLIP-2 training path could be made to run end to end in the local environment and produce valid outputs at every stage.

The next major experiment was the office-scale run on 50k / 1k / 1k data with a 3 / 3 / 5 schedule. This run was designed to test the most important local hypothesis: whether increasing both dataset size and optimization budget, while keeping the architecture fixed, would materially improve caption quality. The answer was yes. Caption diversity increased sharply, BLEU/CIDEr improved substantially, and the outputs were no longer domi-

nated by a tiny set of repeated generic captions.

After that, I ran a caption-only polish experiment from the office-scale stage-2 checkpoint with per-epoch checkpoint saving. The motivation was methodological cleanliness rather than architectural novelty: preserve every epoch of the caption run so that best-epoch selection would not depend on the final checkpoint alone. That rerun completed cleanly, but the earlier office epoch-3 snapshot remained the best validation checkpoint.

The final long-run desktop experiment pushes the same architecture to a 100k / 1k / 1k split with a 10 / 10 / 15 schedule. It keeps CLIP-L, OPT-350M, image resolution 224, and the local evaluation path fixed in order to isolate the effect of increased training data and optimization budget. That run completed successfully and produced a full set of per-epoch checkpoints and prediction files, which made it possible to compare the best validation epoch against the last epoch directly.

The local optimization details were also kept intentionally simple and consistent. Stage 1 and stage 2 use an initial learning rate of 10^{-4} with linear warmup and cosine decay, while caption finetuning uses 10^{-5} . Weight decay is 0.05 throughout. Caption decoding during validation uses beam size 3, maximum length 30, and minimum length 8. These values are not claimed to be optimal. Instead, they reflect a stability-first philosophy: remove unnecessary degrees of freedom, establish a repeatable baseline, and then scale the parts of the setup that are most likely to matter, namely data and total optimization budget.

Experiment	Data scale	Stage 1	Stage 2	Caption	Image size	Purpose
Initial baseline	10k / 1k / 1k	1 epoch	1 epoch	1 epoch	224	Prove the full BLIP-2 path can run end to end locally
Office-scale run	50k / 1k / 1k	3 epochs	3 epochs	5 epochs	224	Measure the effect of larger data and more optimization
Caption polish rerun	50k / 1k / 1k	reused	reused	5 epochs	224	Preserve per-epoch caption checkpoints and select the best epoch cleanly
Student long run	100k / 1k / 1k	10 epochs	10 epochs	15 epochs	224	Push the best student-scale result further while holding architecture fixed

Table 4: Main experiment schedule progression in the local reproduction study.

3.8 Results and Evaluation

3.8.1 Completed Training Outcomes

The full local pipeline completed successfully on the 10k baseline, the stronger office-scale 50k schedule, and the final student-scale 100k long run, and all three stages produced valid checkpoints. This is already a meaningful outcome because it shows that the staged BLIP-2 method can be executed end to end on local hardware after the necessary engineering adjustments. The strongest completed stage-wise training losses were:

- Stage 1 office run: train loss 0.412, with ITC 0.214, ITM 0.054, and LM 3.031
- Stage 2 office run: train loss 0.767
- Caption fine-tuning: epoch-3 train loss 0.367 on the best-metric office run, and 0.362 at epoch 4 on the clean polish rerun
- Student long run stage 1: final train loss 0.330, with ITC 0.365, ITM 0.037, and LM 2.241
- Student long run stage 2: final train loss 0.363
- Student long run captioning: final train loss 0.365 at epoch 14

3.8.2 Captioning Metrics

For the final caption comparison, I ran a reduced evaluation path on the 1,000-image validation subset that computes BLEU and CIDEr while

avoiding the unstable METEOR path on Windows. Because `report_metric=false` was required during the checkpoint-producing runs, the BLEU/CIDEr results were computed afterward from the saved validation prediction files using the official `pycocoevalcap` BLEU and CIDEr implementations with a subset-matched ground-truth file. I report BLEU and CIDEr on the same paper-style $\times 100$ scale used in the BLIP-2 captioning table.

The best local metric vector from a completed run still comes from the office-scale run’s `val_epoch3.json`: BLEU-1 53.86, BLEU-2 33.52, BLEU-3 19.65, BLEU-4 11.13, and CIDEr 37.57. The completed 100k long run did not surpass this earlier result. Its best checkpoint occurred at epoch 2 with BLEU-4 8.57 and CIDEr 25.34, while its final epoch-14 checkpoint degraded to BLEU-4 6.61 and CIDEr 18.36. Table 5 compares the directly comparable local best metrics against the paper’s reported COCO captioning result for BLIP-2 ViT-g OPT2.7B.

The completed results are also useful when compared internally across local runs. The 10k baseline reached only BLEU-4 1.45 and CIDEr 3.03 and produced only 61 unique captions over 1,000 validation images. The completed 50k office run improved this sharply to BLEU-4 11.13 and CIDEr 37.57 with 563 unique captions. The completed 100k long run improved diversity further to 677 unique captions at its best epoch, but it did not improve BLEU/CIDEr and in fact peaked early before

Metric	Paper	Local
BLEU-4	43.7	11.13
CIDEr	145.8	37.57

Table 5: Captioning comparison between the BLIP-2 paper’s COCO fine-tuned ViT-g OPT2.7B result and the best completed local office-scale reduced-subset reproduction. The numbers are not directly comparable because the local run uses a smaller model, a smaller dataset, lower image resolution, and a different evaluation split.

degrading over later epochs. This means that, under the fixed local architecture used here, more data plus a much longer schedule did not automatically translate into better caption metrics.

3.8.3 Best Checkpoint Selection During Training

Yes, it is possible to select the best checkpoint during training rather than simply taking the last epoch, and in this project that distinction turned out to matter. Because the local Windows caption runs used `report_metric=false` to avoid unstable online METEOR scoring, each caption epoch saved both a checkpoint and a corresponding `val_epoch*.json` prediction file. After training, I evaluated every saved validation epoch offline on the same 1,000-image subset and selected the best checkpoint by validation CIDEr, using BLEU-4 and caption diversity as supporting diagnostics rather than as the primary selector.

This procedure is especially important for the completed 100k long run. If I had taken the final checkpoint by default, the reported result would have been BLEU-4 6.61 and CIDEr 18.36. Instead, selecting the best validation epoch recovers epoch 2 at BLEU-4 8.57 and CIDEr 25.34. Table 7 makes this explicit. The broader lesson is that under long student-scale runs, the best caption checkpoint may occur well before the final epoch, so model selection should be metric-driven rather than endpoint-driven.

3.8.4 Qualitative Behavior

The best local caption file still shows visible degeneration, but it is substantially less collapsed than the earlier small-run baseline. Out of 1,000 predictions, there are 563 unique captions. The most common caption appears 27 times, the top 5 captions account for 113 outputs, and the top 10 account for 176 outputs. This means the model remains weaker than the paper, but it is no longer

trapped in the severe low-diversity regime seen in the earlier reduced run.

Table 8 shows representative examples. The generated captions are often generic object-centric or person-centric templates that only partially match the image content. This pattern is important because it shows that the local model is not failing randomly; it is falling into a semantically narrow but linguistically plausible caption regime.

3.8.5 100k Run Outcome

The long-run 100k / 1k / 1k desktop experiment completed successfully under the full 10 / 10 / 15 schedule and produced checkpoints through epoch 14. The run demonstrates that the local pipeline can sustain a much longer optimization budget on a larger subset without crashing. However, the metric outcome is mixed. Relative to the office-scale best checkpoint, the 100k run increased caption diversity but reduced BLEU/CIDEr. The best checkpoint occurred very early, at epoch 2, and later epochs drifted downward. As a result, the final experimental conclusion is not “more data always helped,” but rather “more data and more epochs helped only up to a point under this architecture and optimizer setup.”

3.9 Failure Analysis

The reproduction outcome is best described as *pipeline success but model-quality failure*. I successfully reproduced the main BLIP-2 stages, obtained all checkpoints, and generated captions from the final fine-tuned model. However, the completed local model is still far below the paper’s quality. The main reasons are:

- **Massive scale mismatch.** The paper uses 129M pre-training images and much larger backbones; the completed local runs use only reduced COCO Karpathy subsets and OPT-350M.
- **Optimization-budget mismatch.** The earliest successful local run used only one epoch per stage. Later local improvements came primarily from longer schedules and larger subsets, which strongly suggests that under-training was a major contributor to collapse.
- **Backbone mismatch.** The paper’s best OPT caption result uses ViT-g and OPT-2.7B. The local run uses CLIP-L and OPT-350M.

Run	Scale	Selection	BLEU-4	CIDEr	Unique caps
10k baseline	10k	epoch 0 / final	1.45	3.03	61
50k office best	50k	best epoch 3	11.13	37.57	563
50k polish completion	50k	final epoch 4	10.49	37.14	607
100k student long best	100k	best epoch 2	8.57	25.34	677
100k student long final	100k	final epoch 14	6.61	18.36	734

Table 6: Comparison across completed local runs, reported on the paper-style $\times 100$ metric scale.

100k checkpoint	BLEU-4	CIDEr	Unique caps
Best epoch 2	8.57	25.34	677
Final epoch 14	6.61	18.36	734

Table 7: Best-versus-final checkpoint comparison for the completed 100k long run.

- **Evaluation mismatch.** The paper reports Karpathy COCO test results with standard caption metrics. The local run uses a reduced validation subset and a partially simplified scorer path.
- **Environment friction.** Windows-specific issues in the COCO evaluation pipeline forced practical compromises during the final checkpoint run.

These points explain why the gap should not be interpreted as a mysterious implementation failure. The paper-performance gap is the expected outcome of a faithful but aggressively downscaled reproduction. In fact, one of the most useful findings of the project is that the gap narrows in a sensible way when the local training setup is strengthened from 10k to 50k. At the same time, the completed 100k run shows that simply extending both data scale and epoch count does not guarantee continued improvement. That pattern supports the claim that the local system is behaving like a weaker but structurally valid BLIP-2 pipeline rather than a fundamentally broken model: it improves in interpretable ways, peaks at a meaningful checkpoint, and then appears to overtrain or drift under a longer schedule.

That said, the reproduction is still meaningful even without parity. It validates the engineering path, demonstrates how BLIP-2 can be down-scaled into a student-sized experiment, and produces enough evidence to evaluate the architecture’s behavior under tight compute. The failure, therefore, is not a failure of the project; it is the central empirical result of the project.

4 Threats to Validity

Several threats to validity limit how strongly the local results can be compared to the original paper. First, the local evaluation uses reduced validation subsets rather than the full Karpathy COCO test split reported in the paper. This makes the internal comparison across local runs valid, but weakens any claim of direct paper-level comparability. Second, the local architecture differs materially from the strongest published BLIP-2 captioning configuration because it replaces ViT-g and OPT-2.7B with CLIP-L and OPT-350M. Third, the project applies a small number of code patches to make the official LAVIS path work in a single-process Windows environment. These patches were necessary for reproduction, but they also mean the final system is not a completely untouched upstream implementation.

The project also contains a systems-validity issue that is easy to overlook. Because the reproduction required practical engineering fixes, some of the measured difficulty reflects not only BLIP-2 as a scientific method but also LAVIS as a software artifact under Windows. This is still valuable evidence, but it should be interpreted carefully: part of what is being reproduced here is a research codebase in a non-native environment, not just a mathematical model.

There is also a model-selection validity issue. The best local caption checkpoints are selected by validation CIDEr on the same 1,000-image subset used throughout the study. This is reasonable for a reproduction report, but it means that the paper compares the best local validation snapshot against the published paper result rather than against an untouched held-out test set under identical model-selection rules. The main conclusions are still informative, but they should be read as careful reproduction evidence rather than as benchmark-equivalent leaderboard claims.

Image ID	Reference content (paraphrased from COCO references)	Generated caption
91500	Two teenagers sit on folding chairs and play video games.	a woman with her hands in her pockets
511622	A woman wearing a scarf cooks food in a pan or wok.	a woman with her hands on her hips
341113	A woman in a wetsuit rides a surfboard on a wave.	a man with a gun in his hand
407403	A painting shows yellow tulips in a white vase.	a woman holding a baby in her arms
353027	A hand lifts a slice of mushroom pizza from the pie.	a woman with a baby in her arms
31240	A train is leaving or standing at a station.	a woman with her dog in her lap

Table 8: Representative qualitative failures from the reduced validation subset.

5 Reproducibility and Artifacts

One strength of this project is that the reproduction process is documented as an artifact set rather than as a single narrative result. The repository contains the local configs, stage launchers, data-subset tools, evaluation scripts, run registries, progress registries, checkpoint registries, failure notes, and paper-facing metric summaries needed to inspect or continue the work. In particular, the files under `blip2_repro/configs/`, `blip2_repro/scripts/`, `docs/blip2/`, and `metrics/blip2/` together describe the operational history of the project at a much finer granularity than is possible inside the paper alone.

This artifact trail matters for two reasons. First, it makes the project auditable: the reported caption metrics can be traced back to specific saved prediction files and subset-matched ground-truth files. Second, it makes the work extensible: later runs, including the completed 100k long-run desktop pipeline, are direct continuations of the same codebase rather than separate ad hoc experiments. The project therefore contributes not only a result, but a reusable student-scale BLIP-2 reproduction scaffold. From a thesis-style perspective, this is important because the scientific contribution is partly methodological and infrastructural, not only numerical.

6 Conclusion

Among the three selected papers, BLIP-2 was the best implementation target because it is modular, open, and still intellectually coherent after aggressive downscaling. LLaVA is a strong alternative when moderate cluster compute is available, while VisionLLM v2 is too heavy for this assignment without major institutional infrastructure.

The local BLIP-2 reproduction successfully completed stage-1 representation learning, stage-2 generative alignment, and caption fine-tuning on a sin-

gle RTX 3070 8GB GPU. This satisfies the implementation requirement as a non-trivial reproduction of the core method. However, the completed local results do not approach the paper’s reported captioning quality. The quantitative and qualitative evidence instead shows that the model remains substantially weaker than the published BLIP-2 captioning system, even after meaningful improvement from the 10k baseline to the 50k office run.

The completed 100k long run sharpens that conclusion rather than overturning it. It shows that more data and a much longer schedule can increase caption diversity and sustain the pipeline at larger scale, but they do not automatically yield better caption metrics under a fixed student-scale architecture. In this project, the best 100k checkpoint still underperformed the earlier 50k office best, and the last 100k checkpoint was worse still. Therefore, model selection by best validation metric is essential, and the final result should be understood as the best checkpoint reached during training rather than the last checkpoint saved.

Therefore, the main conclusion of this project is twofold. First, BLIP-2’s *pipeline* is reproducible at small scale. Second, BLIP-2’s *performance* depends strongly on the scale of the frozen backbones, pre-training data, fine-tuning budget, model-selection strategy, and evaluation environment. That contrast is itself a useful research lesson, and it helps explain why paper-level multimodal performance remains difficult to reproduce faithfully on commodity hardware.

References

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Maria Monteiro, Jacob Menick, Sebastian Borgeaud, and 8 oth-

ers. 2022. Flamingo: a visual language model for few-shot learning. *arXiv preprint arXiv:2204.14198*.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven C. H. Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. *arXiv preprint arXiv:2301.12597*.

Junnan Li, Dongxu Li, Caiming Xiong, and Steven C. H. Hoi. 2022. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning*.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollar, and C. Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*.

Jiannan Wu, Muyan Zhong, Sen Xing, Zeqiang Lai, Zhaoyang Liu, Zhe Chen, Wenhai Wang, Xizhou Zhu, Lewei Lu, Tong Lu, Ping Luo, Yu Qiao, and Jifeng Dai. 2024. Visionllm v2: An end-to-end generalist multimodal large language model for hundreds of vision-language tasks. *arXiv preprint arXiv:2406.08394*.